

Team Reference Document

NCPC 2026 – Nordic Collegiate Programming Contest

3 October 2026 — IT-Universitetet i København

Judge: Kattis. C++ compiled with `g++-15 15.2.0, -g -O2 -std=gnu++23 -static`. Python 3 is pypy3 7.3.23 (Python 3.11.15).

Rules (2026-09-28): printed material unlimited; *no* digital prewritten code, no textbook PDFs on the machine, no GenAI/LLM tools. Live web limited to the contest page, cpcreference.com and docs.python.org (built-in search only).

Scoring: rank by problems solved; ties by penalty (minutes to first AC, +20 min per wrong submission *on solved problems*). Wrong submissions on unsolved problems are free.

The problems are not sorted by difficulty.

Contents

Part I – Checklists	2
1 Complexity budget — pick the algorithm before you write it	2
1.1 Reading the constraint backwards	2
1.2 Constant-factor reality checks	2
1.3 Overflow budget	2
2 The judge said no. Now what?	3
2.1 Compile Error	3
2.2 Wrong Answer	3
2.3 Time Limit Exceeded	3
2.4 Run Time Error	3
2.5 Memory Limit Exceeded	4
2.6 Before every submission — 15-second ritual	4
2.7 When to walk away	4
3 Development workflow	5
3.1 Typical loop on one problem	5
3.2 Three ways to feed input	5
3.3 Debug output	5
3.4 Stress testing — finding the bug you cannot see	5
3.5 Sanitizers, honestly	6
3.6 Python	6
4 Contest day — NCPC 2026	7
4.1 The night before	7
4.2 What you may bring	7
4.3 10:00 — doors open	7
4.4 11:00–11:20 — read everything, write nothing	7
4.5 11:20 — first commitment	7
4.6 11:30 onwards — use the scoreboard as a difficulty oracle	7
4.7 Role split for two people	8
4.8 Pacing	8
4.9 Penalty maths — worth understanding	8
4.10 Realistic expectations	8
4.11 If something breaks	8
Part II – Code templates	9
4.12 templates/cpp/solution.cpp	9
4.13 templates/cpp/binary_search.cpp	10
4.14 templates/cpp/prefix_sums.cpp	11
4.15 templates/cpp/ds/dsu.cpp	12
4.16 templates/cpp/ds/fenwick.cpp	12
4.17 templates/cpp/ds/segment_tree.cpp	13
4.18 templates/cpp/graphs/traversal.cpp	14
4.19 templates/cpp/graphs/dijkstra.cpp	15
4.20 templates/cpp/graphs/algorithms.cpp	16
4.21 templates/cpp/dp/classic.cpp	18
4.22 templates/cpp/math/number_theory.cpp	19
4.23 templates/cpp/strings/strings.cpp	21
4.24 templates/cpp/geometry/geometry.cpp	22
4.25 templates/python/solution.py	24

Part I – Checklists

1 Complexity budget — pick the algorithm before you write it

Kattis time limits are typically **1–4 s**. Assume roughly **10^8 simple operations per second** for C++ at -O2. Read n from the constraints section, find the row, and that tells you what you are allowed to write.

max n	Budget	What fits	Typical technique
≤ 10	$O(n!)$	3.6M	brute-force all permutations
≤ 20	$O(2^n \cdot n)$	20M	bitmask / subset enumeration
≤ 100	$O(n^3)$	1M	Floyd–Warshall, triple loop, small DP
$\leq 1\,000$	$O(n^2)$	1M	pairwise loops, 2D DP, $O(n^2)$ graph
$\leq 10^5$	$O(n \log n)$	1.7M	sort, set/map, binary search, Dijkstra
$\leq 10^6$	$O(n)$	1M	prefix sums, two pointers, single sweep
$\leq 10^9$	$O(\log n) / O(\sqrt{n})$	—	binary search on answer, math, trial division
$\geq 10^{18}$	$O(1)$	—	closed-form formula only

The single most useful inference: $n \leq 10^5$ almost always means $O(n \log n)$ — i.e. *sort it, or put it in a set/map, or binary search it*. $n \leq 10^6$ means you get one linear pass and nothing more.

1.1 Reading the constraint backwards

If you cannot see the intended algorithm, the constraint tells you the shape:

- $n \leq 20 \rightarrow$ the answer is a subset or permutation search. Stop looking for a clever polynomial algorithm.
- $n \leq 5000$ with a 2D feel $\rightarrow O(n^2)$ DP.
- Sum of n over all test cases bounded $\rightarrow$ your per-case cost must be near-linear.
- Answer is "minimum x such that..." / "maximum x such that..." $\rightarrow$ **binary search on the answer**; you only need a feasibility check.
- Huge coordinate range but few points $\rightarrow$ coordinate compression.

1.2 Constant-factor reality checks

These matter more than beginners expect:

Thing	Cost
cin/cout without <code>sync_with_stdio(false)</code>	up to $10\times$ slower — always disable
endl instead of '\n'	flushes every line; can alone cause TLE
map / set	$\sim 5\text{--}10\times$ slower than <code>unordered_*</code> and than sorting a vector
unordered_map	$O(1)$ average but $O(n)$ worst case under anti-hash tests
recursion	$\sim 2\text{--}3\times$ a loop, and stack-overflows near $1e5$ depth
% and / on long long	several times the cost of int

1.3 Overflow budget

Type	Max
int	$\approx 2.1 \times 10^9$
long long	$\approx 9.2 \times 10^{18}$
double	exact integers only to $2^{53} \approx 9 \times 10^{15}$

Rule: if any intermediate value can exceed $\sim 2 \times 10^9$, use `long long` everywhere in that expression. $n * (n-1) / 2$ overflows int at $n \approx 65\,536$ — far below a typical $n \leq 10^5$ bound. This is the most common single cause of wrong answers at beginner level.

Note `1 << 31` overflows; write `1LL << 31`.

2 The judge said no. Now what?

Work top to bottom. **Do not resubmit without changing something specific** — every wrong submission on a problem you eventually solve costs 20 penalty minutes.

2.1 Compile Error

Almost always a missing header or a C++ version issue. `bits/stdc++.h` works on the judge (GCC), so this is usually a typo or a local-only construct.

2.2 Wrong Answer

Check in this order — cheapest and most likely first.

1. **Did you read the output format exactly?** Re-read the output section verbatim. Singular vs plural, Case #1:, capitalisation, trailing text.

2. **Overflow.** Could any value exceed 2×10^9 ? Make it `long long`.

Check *intermediates*: `a*b` overflows even if the answer is small.

3. **Did you handle $n = 0$, $n = 1$, or an empty input?** Single-element and empty cases break more solutions than any other edge case.

4. **Off-by-one.** 0-indexed vs 1-indexed. `<` vs `<=`. Inclusive ranges.

5. **Are you resetting state between test cases?** If the input has multiple cases, clear every global array/vector/counter.

6. **Floating point.** Did you print with enough precision?

`cout << fixed << setprecision(10)`. Never compare doubles with `==`; use `fabs(a-b) < 1e-9`. If the problem is exact, avoid doubles entirely.

7. **Did you misread the problem?** Re-read the statement from the top.

At beginner level this is the single most common cause. Have your teammate read it independently and state the task back to you.

8. **Ties / multiple valid answers.** Does the problem say "any valid answer"? If not, there is a specific tie-break rule you may have missed.

9. **Run the sample by hand.** Not with your code — on paper. If your manual answer differs from the expected output, you misread the problem.

2.3 Time Limit Exceeded

1. `ios::sync_with_stdio(false); cin.tie(nullptr);` — present? Up to $10 \times$.

2. `endl` → `'\n'`. `endl` flushes; in a loop it alone causes TLE.

3. **Recompute your complexity against the real constraints.** See

`complexity-budget.md`. Did you assume $n \leq 1000$ when it is 10^5 ?

4. **Accidental $O(n^2)$:** `v.erase(v.begin())` in a loop, `string += building` a huge string, passing containers **by value** instead of `const &`.

5. **unordered_map worst case.** Under anti-hash tests it degrades to $O(n)$. Swap to `map`, or sort a `vector<pair<>>`.

6. **Infinite loop.** A `while` whose condition never changes; a graph cycle without a visited check.

2.4 Run Time Error

1. **Array/vector out of bounds** — by far the most common. Rebuild with `make d F=yourfile (AddressSanitizer + _GLIBCXX_DEBUG)`; it names the exact line.

2. **Stack overflow from deep recursion.** $\sim 10^5$ frames is the practical limit. Rewrite iteratively.

3. **Division or modulo by zero.**

4. `.front()`, `.top()`, `.back()` on an empty container.

5. **Reading past end of input** — check whether `cin >>` actually succeeded.

2.5 Memory Limit Exceeded

1. A 2D array sized by the *maximum* constraint rather than the actual input.

`int dp[1000][1000]` is 4 MB; `[10000][10000]` is 400 MB.

2. Declare large arrays globally, not on the stack.
 3. Do you need all of the DP table, or only the previous row?
-

2.6 Before every submission — 15-second ritual

- ☐ Compiled with **zero warnings** (`-Wall -Wextra`)
- ☐ Passes **every** provided sample, byte for byte (`make & F=...`)
- ☐ Tested `n = 1` and the smallest legal input
- ☐ Every value that could exceed `2e9` is `long long`
- ☐ No leftover debug printing to `stdout`
- ☐ `'\n'`, not `endl`

2.7 When to walk away

If a problem has eaten **45 minutes with no accepted submission**, switch to a different one and come back. The scoreboard is the better guide to what is solvable than your own attachment to the problem you already started.

3 Development workflow

One command per thing you actually do during a contest. Solutions live in the repo root, one file per problem: a.cpp, b.cpp, ... k.cpp.

```
make new P=a      # create a.cpp from the template
make run P=a      # build + run (a.in -> a.out if a.in exists)
make test P=a     # diff against a.1.in/a.1.ans, a.2.in/a.2.ans, ...
make debug P=a    # bounds-checked build; finds out-of-bounds
make brut P=a     # scaffold a_brut.cpp + gen.py
make stress P=a   # random tests until brute force disagrees
make submit P=a   # judge-flag build + sample check before uploading
make clean P=a
```

All scratch files (a.cpp, a.*.in, gen.py, in.txt, binaries) are gitignored, so the repo stays clean between contests.

3.1 Typical loop on one problem

```
make new P=a      # start
pbpaste > a.1.in  # paste the sample input from the problem page
# ...write the expected output into a.1.ans...
make test P=a     # must pass before you even think about submitting
make submit P=a   # rebuilds with judge flags, re-runs samples
```

make test compares byte for byte. If it prints FAIL with identical-looking strings, you have a trailing space or newline difference.

3.2 Three ways to feed input

All three work; use whichever is fastest in the moment.

How	When
make run P=a with a.in present	the normal case
./a < a.in > a.out	manual control
put data in in.txt, just run ./a	pressing Run in an editor, no shell

The in.txt path works because solution.cpp calls freopen — but only inside `#ifdef LOCAL`, and only if in.txt actually exists. The judge build never defines LOCAL, so a stray freopen cannot reach Kattis. This matters: an unguarded freopen in a submission makes the judge read a file that does not exist, and every test fails for a reason that looks nothing like the cause.

3.3 Debug output

dbg(...) prints to **stderr**, which Kattis ignores, and compiles to nothing without `-DLOCAL`:

```
dbg(n, a); // [90] (n, a): 3 {1, 2, 3}
```

It formats vectors, pairs, sets and maps automatically.

Why this matters. The widely copied dbg_out macro writes to cout.

That injects debug text into the answer the judge reads — a silent Wrong

Answer that looks like an algorithmic bug. You will not notice locally,

because locally you are reading your own output.

3.4 Stress testing — finding the bug you cannot see

When a solution passes the samples but gets Wrong Answer, and you cannot see why, stop reading the code and let the computer find a counterexample. This is the standard olympiad technique:

```
gen.py SEED -> random small test
a_brut.cpp -> slow solution that is OBVIOUSLY correct
a.cpp -> your fast solution
diff -b -> stop at the first disagreement
```

```
make brut P=a      # creates a_brut.cpp and gen.py
# ...write the O(n^2) or O(n!) version in a_brut.cpp...
# ...edit gen.py so it emits this problem's input format...
make stress P=a    # or: make stress P=a N=1000
```

Output on failure:

```
MISMATCH on seed 1 (reproduce: python3 gen.py 1)
--- input (stress.in) ---
3
10 2 5
--- yours ---
12
--- brute force ---
17
```

Keep the generator **small** — $n \leq 8$, values ≤ 10 . A counterexample you can read in three seconds is worth far more than a correct-but-huge one, and the brute force has to finish quickly.

3.5 Sanitizers, honestly

`make debug` uses `_GLIBCXX_DEBUG`, not `AddressSanitizer`. On macOS:

- Homebrew GCC ships **no** `libasan` or `libubsan`, so `-fsanitize=` cannot link.
- Apple clang has the sanitizers but has **no** `<bits/stdc++.h>` and no

`__gnu_pbds`, so it cannot compile the template at all.

`_GLIBCXX_DEBUG` catches the dominant contest bug — container indexing — and reports it precisely:

```
Error: attempt to subscript container with out-of-bounds index 9,  
but container only holds 3 elements.
```

3.6 Python

```
python3 sol.py < a.in > a.out      # normal  
LOCAL=1 python3 sol.py            # reads in.txt, writes out.txt, dbg() on
```

Remember Kattis’s “Python 3” is **PyPy 7.3.23 (Python 3.11.15)**. Loops are fast; numpy is not available. Do not use 3.12+ syntax.

4 Contest day — NCPC 2026

Saturday 3 October 2026 · doors 10:00 · contest 11:00–16:00 CEST IT-Universitetet i København, Rued Langgaards Vej 7, 2300 København S (site for ITU + DTU + KU). Food during, pizza and beer after.

Also runs as **DM i programming** — the Danish championship.

4.1 The night before

- ☐ Confirm the team is registered and you have **Kattis team credentials**

(a team account, *not* your personal Kattis login). The contest lives at `ncpc26.kattis.com`.

- ☐ Print everything. See `reference/INDEX.md`.
- ☐ **Delete the reference PDFs from the laptop.** Printed = allowed;

the digital file on disk is exactly what the 2026-09-28 rule change bans.

- ☐ Disable any AI/LLM editor plugin (Copilot, Codeium, Cursor tab). These

are explicitly disallowed. Turn them off at the settings level, not just visually.

- ☐ `make check-toolchain` — confirm `g++-15` is what builds your code.
- ☐ Charger, mouse, laptop. One computer, one keyboard, one mouse, one screen.
- ☐ Sleep. Genuinely higher value than another hour of cramming.

4.2 What you may bring

Allowed	Banned
Unlimited printed material	Any digital prewritten code
Textbooks, team reference docs	PDFs of textbooks on the machine
One laptop, one keyboard/mouse/screen	Phones and any extra device
<code>cppreference.com</code> , <code>docs.python.org</code> (built-in search only)	Any other website
Basic editor autocomplete	Any GenAI/LLM tool or plugin

4.3 10:00 — doors open

Arrive at 10:00, not 10:55. Set up, log in, open the scoreboard and the submit page, and leave them open. Put the printed references where both of you can reach them.

4.4 11:00–11:20 — read everything, write nothing

The problems are **not sorted by difficulty** — this is stated verbatim in every NCPC booklet. Letter order tells you nothing.

With two people:

- Person A reads $A \rightarrow K$. Person B reads $K \rightarrow A$.
- Score each problem on a sheet: **E** (I see the solution now),

M (I see the shape, needs thought), **H** (no idea).

- Do **not** start coding during this window, even if problem C looks trivial.

4.5 11:20 — first commitment

Start the problem you both scored **E** with the shortest implementation. Not the most interesting one. Shortest.

4.6 11:30 onwards — use the scoreboard as a difficulty oracle

With 230+ teams competing, the easy problems light up within 15–20 minutes. **Check the scoreboard every ~20 minutes.** If 40 teams have solved problem G and you had it as H, re-read G — you misjudged it. This signal is far more reliable than a beginner's own read of a statement.

4.7 Role split for two people

One keyboard means the bottleneck is thinking, not typing.

- **Coder** writes the current solution.
- **Reader** does *not* watch the screen. The reader works the next problem on

paper: edge cases, approach, the complexity check from `complexity-budget.md`. Hand over a written plan, then swap roles.

- When a submission comes back WA, the **reader** runs `debug.md` out loud

while the coder re-reads the statement. Two people, two different inputs.

4.8 Pacing

Time	Target	If you are behind
12:00	1 solved	Drop to the problem with the most scoreboard solves
13:00	2–3 solved	Stop any problem older than 45 min
14:00	3–4 solved	Pick purely by scoreboard count now
15:00	4–5 solved	Finish what is nearly done; start nothing new
15:45	—	Last realistic submit window

4.9 Penalty maths — worth understanding

Penalty = (minutes to first accepted) + $20 \times$ (wrong submissions **on solved problems**). Wrong submissions on problems you never solve are **free**.

Two consequences:

1. Never leave a problem unsubmitted at the end out of fear. A speculative

submission at 15:55 costs nothing if it fails.

2. During the contest, solving one more problem always beats optimising

penalty. Rank is by count first; penalty only breaks ties.

4.10 Realistic expectations

Across 2021–2025, the median NCPC team solved **3–5 of 11**, and 15–25% of all teams solved 0 or 1. Every year has 1–2 problems solved by **zero or one** team out of 230+. You are not expected to solve the hard half. Solve the easy half cleanly and you finish mid-table or better.

4.11 If something breaks

Submit a **clarification request** through the judge system. Expect "No comment, read problem statement" — that answer means your reading is correct and there is no ambiguity, which is itself useful information.

Part II – Code templates

4.12 templates/cpp/solution.cpp

```

1 // Base solution template – NCPC 2026.
2 //
3 // make new P=a      create work/a.cpp from this file
4 // make run P=a      build + run with work/a.in -> work/a.out
5 // make test P=a     run every work/a.*.in against work/a.*.ans
6 // make debug P=a    sanitizers + bounds-checked STL
7 // make stress P=a   random tests vs a brute force
8 //
9 // -----
10 // WHY dbg() WRITES TO cerr, NOT cout
11 //
12 // A debug macro that prints to cout writes into the answer the judge reads.
13 // It will not look wrong locally, because locally you read your own output.
14 // On Kattis it is a silent Wrong Answer, and you will burn submissions
15 // hunting an algorithmic bug that is not there.
16 //
17 // cerr is a separate stream; Kattis ignores it. And because dbg is wrapped
18 // in #ifdef LOCAL, it compiles to literally nothing unless you build with
19 // -DLOCAL. The Makefile passes -DLOCAL for local runs and never for the
20 // file you submit, so leaving dbg() calls in your submission is harmless.
21 // -----
22 #include <bits/stdc++.h>
23
24 // Policy-based data structures: ordered_set supports "how many elements are
25 // less than x" and "what is the k-th element" in O(log n).
26 // GCC only — these headers do not exist on Apple clang.
27 #include <ext/pb_ds/assoc_container.hpp>
28 #include <ext/pb_ds/tree_policy.hpp>
29
30 using namespace std;
31 using namespace __gnu_pbds;
32
33 template <class T>
34 using ordered_set =
35     tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
36
37 using ll = long long;
38 using pii = pair<int, int>;
39 using pll = pair<ll, ll>;
40 using vi = vector<int>;
41 using vll = vector<ll>;
42
43 #define pb push_back
44 #define eb emplace_back
45 #define all(v) (v).begin(), (v).end()
46 #define rall(v) (v).rbegin(), (v).rend()
47 #define sz(v) ((int)(v).size())
48
49 // endl is a function that FLUSHES. In a loop that alone can cause TLE.
50 // Redefining it to a plain newline removes the flush everywhere at once.
51 #define endl '\n'
52
53 // ----- debug output -----
54 #ifdef LOCAL
55 template <class T, class = decltype(begin(declval<T>()))>,
56         class = enable_if_t<!is_same_v<T, string>>>
57 ostream &operator<<(ostream &os, const T &c);
58 template <class A, class B>
59 ostream &operator<<(ostream &os, const pair<A, B> &p) {
60     return os << '(' << p.first << ", " << p.second << ')';
61 }
62 template <class T, class, class>
63 ostream &operator<<(ostream &os, const T &c) {
64     os << '{';
65     for (auto it = begin(c); it != end(c); ++it)
66         os << (it == begin(c) ? "" : ", ") << *it;
67     return os << '}';
68 }
69
70 inline void dbg_out() { cerr << '\n'; }
71 template <class Head, class... Tail>
72 void dbg_out(Head H, Tail... T) {
73     cerr << ' ' << H;
74     dbg_out(T...);
75 }
76 #define dbg(...) \
77     cerr << "[" << __LINE__ << "] (" << #__VA_ARGS__ << "):", dbg_out(__VA_ARGS__)
78 #else
79 #define dbg(...) ((void)0)
80 #endif
81 // -----
82
83 void solve() {
84     int n;
85     cin >> n;
86
87     vll a(n);
88     for (ll &x : a) cin >> x;
89
90     dbg(n, a); // vanishes unless built with -DLOCAL
91
92     cout << accumulate(all(a), 0LL) << endl;
93 }
94
95 int main() {
96     ios_base::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99 #if defined(LOCAL) && defined(USE_FILES)

```

```

100 // Opt-in file I/O: `make runf P=a` passes -DLOCAL -DUSE_FILES and reads
101 // in.txt / writes out.txt. Plain `make run P=a` uses shell redirection.
102 //
103 // This is deliberately NOT automatic. Auto-detecting in.txt would
104 // silently override `./sol < a.in`, which is exactly the kind of
105 // surprise you cannot afford mid-contest.
106 //
107 // Neither LOCAL nor USE_FILES is ever defined by a judge -- note that
108 // Kattis does NOT define ONLINE_JUDGE, so the common #ifndef ONLINE_JUDGE
109 // guard silently leaves freopen active on Kattis. Use LOCAL.
110 if (!freopen("in.txt", "r", stdin)) {
111     cerr << "in.txt not found\n";
112     return 1;
113 }
114 if (!freopen("out.txt", "w", stdout)) {
115     cerr << "cannot open out.txt for writing\n";
116     return 1;
117 }
118 #endif
119
120 // Multi-test input? Replace the body with:
121 // int t; cin >> t; while (t--) solve();
122 solve();
123 return 0;
124 }

```

4.13 templates/cpp/binary_search.cpp

```

1 // Binary search on the answer – the highest-value beginner technique.
2 //
3 // WHEN TO REACH FOR THIS: the problem says "minimum x such that ..." or
4 // "maximum x such that ...", and checking a FIXED x is easy. You never have
5 // to construct the answer, only test candidates. A constraint like
6 // x <= 1e18 with an easy feasibility check is the tell.
7 //
8 // TRAPS:
9 // * lo + (hi-lo)/2, not (lo+hi)/2 – the latter overflows.
10 // * Get the invariant right: below, check() is monotone false...false,
11 //   true...true, and we find the FIRST true.
12 // * For doubles, iterate a fixed ~100 times instead of comparing.
13 #include <bits/stdc++.h>
14 using namespace std;
15
16 using ll = long long;
17
18 // First value in [lo, hi] where check is true. Returns hi+1 if never true.
19 // Requires check to be monotone: once true, true forever after.
20 ll firstTrue(ll lo, ll hi, const function<bool(ll)> &check) {
21     ll result = hi + 1;
22     while (lo <= hi) {
23         ll mid = lo + (hi - lo) / 2; // overflow-safe
24         if (check(mid)) {
25             result = mid;
26             hi = mid - 1;
27         } else {
28             lo = mid + 1;
29         }
30     }
31     return result;
32 }
33
34 // Last value in [lo, hi] where check is true. Returns lo-1 if never true.
35 ll lastTrue(ll lo, ll hi, const function<bool(ll)> &check) {
36     ll result = lo - 1;
37     while (lo <= hi) {
38         ll mid = lo + (hi - lo) / 2;
39         if (check(mid)) {
40             result = mid;
41             lo = mid + 1;
42         } else {
43             hi = mid - 1;
44         }
45     }
46     return result;
47 }
48
49 // Real-valued version. 100 iterations is plenty and cannot loop forever.
50 double binarySearchReal(double lo, double hi, const function<bool(double)> &check) {
51     for (int i = 0; i < 100; i++) {
52         double mid = (lo + hi) / 2;
53         if (check(mid)) hi = mid;
54         else lo = mid;
55     }
56     return lo;
57 }
58
59 #ifdef TEST_BINSEARCH
60 int main() {
61     // Classic: smallest x with x*x >= 1e18.
62     ll n = 1'000'000'000'000'000'000LL;
63     ll r = firstTrue(0, 2'000'000'000LL, [&](ll x) { return x * x >= n; });
64     assert(r == 1'000'000'000LL);
65
66     // Integer square root via lastTrue.
67     ll s = lastTrue(0, 2'000'000'000LL, [&](ll x) { return x * x <= 50LL; });
68     assert(s == 7);
69
70     // Never true / always true boundaries.
71     assert(firstTrue(1, 10, [] (ll) { return false; }) == 11);
72     assert(firstTrue(1, 10, [] (ll) { return true; }) == 1);
73     assert(lastTrue(1, 10, [] (ll) { return false; }) == 0);

```

```

74     assert(lastTrue(1, 10, [])(ll) { return true; }) == 10);
75
76     // Overflow guard: a (lo+hi)/2 implementation would wrap on this range.
77     ll big = firstTrue(0, numeric_limits<ll>::max() / 2,
78         [] (ll x) { return x >= 4'611'686'018'427'387'000LL; });
79     assert(big == 4'611'686'018'427'387'000LL);
80
81     // Worked example: minimum capacity to ship all weights within D days.
82     vector<ll> w = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
83     ll D = 5;
84     auto feasible = [&](ll cap) {
85         if (cap < *max_element(w.begin(), w.end())) return false;
86         ll days = 1, cur = 0;
87         for (ll x : w) {
88             if (cur + x > cap) days++, cur = 0;
89             cur += x;
90         }
91         return days <= D;
92     };
93     assert(firstTrue(1, 55, feasible) == 15);
94
95     double d = binarySearchReal(0, 10, [] (double x) { return x * x >= 2.0; });
96     assert(fabs(d - sqrt(2.0)) < 1e-9);
97     puts("binary search ok");
98 }
99 #endif

```

4.14 templates/cpp/prefix_sums.cpp

```

1 // Prefix sums – answer "sum of range [l, r]" in O(1) after O(n) setup.
2 //
3 // Deceptively high value: many NCPC easy/medium problems reduce to a range
4 // sum, a running maximum, or a difference-array update. NCPC 2025's "km/h"
5 // and "Arithmetic Adaptation" were both running-aggregate problems.
6 //
7 // TRAPS:
8 // * Use long long. Sums of 1e5 values at 1e9 each reach 1e14.
9 // * Decide ONCE whether your range is inclusive [l,r] or half-open [l,r).
10 // Mixing them is the top source of off-by-one bugs here.
11 #include <bits/stdc++.h>
12 using namespace std;
13
14 using ll = long long;
15
16 // 1-indexed prefix sums: p[i] = a[0] + ... + a[i-1], so p[0] == 0.
17 struct Prefix {
18     vector<ll> p;
19
20     explicit Prefix(const vector<ll> &a) : p(a.size() + 1, 0) {
21         for (size_t i = 0; i < a.size(); i++) p[i + 1] = p[i] + a[i];
22     }
23
24     // Inclusive sum over 0-indexed [l, r]. Empty range if l > r.
25     ll sum(int l, int r) const { return l > r ? 0 : p[r + 1] - p[l]; }
26 };
27
28 // 2D prefix sums over a grid.
29 struct Prefix2D {
30     vector<vector<ll>> p;
31
32     explicit Prefix2D(const vector<vector<ll>> &g)
33         : p(g.size() + 1, vector<ll>(g[0].size() + 1, 0)) {
34         for (size_t i = 0; i < g.size(); i++)
35             for (size_t j = 0; j < g[0].size(); j++)
36                 p[i + 1][j + 1] = g[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j];
37     }
38
39     // Inclusive sum over the rectangle (r1,c1)..(r2,c2), 0-indexed.
40     ll sum(int r1, int c1, int r2, int c2) const {
41         return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
42     }
43 };
44
45 // Difference array: apply many range-add updates in O(1) each, then build once.
46 struct Difference {
47     vector<ll> d;
48
49     explicit Difference(size_t n) : d(n + 1, 0) {}
50
51     // Add v to every index in [l, r].
52     void add(int l, int r, ll v) {
53         d[l] += v;
54         d[r + 1] -= v;
55     }
56
57     vector<ll> build() {
58         vector<ll> a(d.size() - 1);
59         ll run = 0;
60         for (size_t i = 0; i < a.size(); i++) a[i] = (run += d[i]);
61         return a;
62     }
63 };
64
65 #ifdef TEST_PREFIX
66 int main() {
67     vector<ll> a = {1, 2, 3, 4, 5};
68     Prefix pre(a);
69     assert(pre.sum(0, 4) == 15);
70     assert(pre.sum(1, 3) == 9);
71     assert(pre.sum(2, 2) == 3);
72     assert(pre.sum(3, 2) == 0); // empty range

```

```

73 // Overflow guard: 1e5 elements of 1e9 sums to 1e14, far past int.
74 vector<ll> big(100000, 1'000'000'000LL);
75 assert(Prefix(big).sum(0, 99999) == 100'000'000'000'000LL);
76
77 vector<vector<ll>> g = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
78 Prefix2D p2(g);
79 assert(p2.sum(0, 0, 2, 2) == 45); // whole grid
80 assert(p2.sum(1, 1, 2, 2) == 28); // 5+6+8+9
81 assert(p2.sum(0, 0, 0, 0) == 1); // single cell
82 assert(p2.sum(0, 1, 1, 2) == 16); // 2+3+5+6
83
84 Difference df(5);
85 df.add(0, 2, 10); // [10,10,10, 0, 0]
86 df.add(1, 4, 1); // [10,11,11, 1, 1]
87 df.add(4, 4, 5); // [10,11,11, 1, 6]
88 assert((df.build() == vector<ll>{10, 11, 11, 1, 6}));
89 puts("prefix ok");
90 }
91 #endif

```

4.15 templates/cpp/ds/dsu.cpp

```

1 // Union-Find / DSU with union by size + path compression.
2 // Near O(1) amortised per operation.
3 //
4 // WHY THIS FILE EXISTS: KACTL deliberately omits plain Union-Find
5 // (content/data-structures/chapter.tex has '% \kactlimport{UnionFind.h}'
6 // commented out, shipping only UnionFindRollback). NCPC's easy/medium graph
7 // problems lean on exactly this. Do not go to the contest without it.
8 //
9 // Typical uses: connected components, cycle detection in an undirected graph,
10 // Kruskal's MST, "are a and b in the same group" queries.
11 #include <bits/stdc++.h>
12 using namespace std;
13
14 struct DSU {
15     vector<int> parent, size_;
16     int components;
17
18     explicit DSU(int n) : parent(n), size_(n, 1), components(n) {
19         iota(parent.begin(), parent.end(), 0);
20     }
21
22     int find(int x) {
23         while (parent[x] != x) {
24             parent[x] = parent[parent[x]]; // path halving
25             x = parent[x];
26         }
27         return x;
28     }
29
30     bool same(int a, int b) { return find(a) == find(b); }
31
32     // Returns false if a and b were already connected.
33     bool unite(int a, int b) {
34         a = find(a);
35         b = find(b);
36         if (a == b) return false;
37         if (size_[a] < size_[b]) swap(a, b);
38         parent[b] = a;
39         size_[a] += size_[b];
40         components--;
41         return true;
42     }
43
44     int compSize(int x) { return size_[find(x)]; }
45 };
46
47 #ifdef TEST_DSU
48 int main() {
49     DSU d(6);
50     assert(d.components == 6);
51     assert(d.unite(0, 1));
52     assert(d.unite(1, 2));
53     assert(!d.unite(0, 2)); // already connected
54     assert(d.same(0, 2));
55     assert(!d.same(0, 3));
56     assert(d.compSize(0) == 3);
57     assert(d.components == 4);
58     d.unite(3, 4);
59     d.unite(4, 5);
60     d.unite(2, 5);
61     assert(d.components == 1);
62     assert(d.compSize(0) == 6);
63     puts("DSU ok");
64 }
65 #endif

```

4.16 templates/cpp/ds/fenwick.cpp

```

1 // Fenwick tree (binary indexed tree) – prefix sums with point updates.
2 // Both operations O(log n). Shorter and faster than a segment tree; reach
3 // for this first when you only need sums.
4 //
5 // WHEN: "update one element, query a range sum", repeatedly. If the array
6 // never changes, use prefix_sums.cpp instead (O(1) queries).
7 //

```

```

8 // TRAPS:
9 // * 1-indexed internally. Public methods below take 0-indexed positions.
10 // * Use long long for sums.
11 // * add() ADDS a delta. To SET a value you must add (new - old).
12 #include <bits/stdc++.h>
13 using namespace std;
14
15 using ll = long long;
16
17 struct Fenwick {
18     int n;
19     vector<ll> t;
20
21     explicit Fenwick(int n) : n(n), t(n + 1, 0) {}
22
23     // a[i] += delta, 0-indexed.
24     void add(int i, ll delta) {
25         for (++i; i <= n; i += i & -i) t[i] += delta;
26     }
27
28     // Sum of a[0..i], 0-indexed inclusive. Returns 0 for i < 0.
29     ll prefix(int i) const {
30         ll s = 0;
31         for (++i; i > 0; i -= i & -i) s += t[i];
32         return s;
33     }
34
35     // Sum of a[l..r], 0-indexed inclusive.
36     ll range(int l, int r) const {
37         return l > r ? 0 : prefix(r) - prefix(l - 1);
38     }
39
40     // Smallest index whose prefix sum is >= target, for non-negative values.
41     // Returns n if no such index. O(log n).
42     int lowerBound(ll target) const {
43         if (target <= 0) return 0;
44         int pos = 0;
45         ll rem = target;
46         for (int pw = 1 << (31 - __builtin_clz(n)); pw > 0; pw >= 1)
47             if (pos + pw <= n && t[pos + pw] < rem) {
48                 pos += pw;
49                 rem -= t[pos];
50             }
51         return pos;
52     }
53 };
54
55 #ifdef TEST_FENWICK
56 int main() {
57     vector<ll> a = {5, 1, 3, 9, 2};
58     Fenwick f((int)a.size());
59     for (int i = 0; i < (int)a.size(); i++) f.add(i, a[i]);
60
61     assert(f.prefix(0) == 5);
62     assert(f.prefix(4) == 20);
63     assert(f.prefix(-1) == 0);
64     assert(f.range(1, 3) == 13);
65     assert(f.range(2, 2) == 3);
66     assert(f.range(3, 2) == 0); // empty
67
68     f.add(2, 7); // a[2]: 3 -> 10
69     assert(f.range(1, 3) == 20);
70     assert(f.prefix(4) == 27);
71
72     // Setting a value means adding the difference.
73     f.add(0, -5 + 100); // a[0]: 5 -> 100
74     assert(f.range(0, 0) == 100);
75
76     // lowerBound over prefix sums 1,3,6,10,15
77     Fenwick g(5);
78     for (int i = 0; i < 5; i++) g.add(i, i + 1);
79     assert(g.lowerBound(1) == 0);
80     assert(g.lowerBound(2) == 1);
81     assert(g.lowerBound(6) == 2);
82     assert(g.lowerBound(15) == 4);
83     assert(g.lowerBound(16) == 5); // beyond the end
84
85     // Overflow guard: 1e5 values of 1e9 sums past int.
86     Fenwick big(100000);
87     for (int i = 0; i < 100000; i++) big.add(i, 1'000'000'000LL);
88     assert(big.prefix(99999) == 100'000'000'000'000LL);
89     puts("fenwick ok");
90 }
91 #endif

```

4.17 templates/cpp/ds/segment_tree.cpp

```

1 // Iterative segment tree – range query, point update. O(log n) both.
2 //
3 // Use when you need a range operation Fenwick cannot do: min, max, gcd, or
4 // any associative combine. For plain sums prefer fenwick.cpp (shorter).
5 //
6 // The iterative form below is far less code than the recursive one and has
7 // no lazy propagation — if you need range UPDATES, that is a different and
8 // much longer structure; think hard about whether you really need it.
9 //
10 // TRAPS:
11 // * query is half-open [l, r). Pick one convention and never mix.
12 // * identity must be neutral for combine: 0 for sum, +inf for min.
13 #include <bits/stdc++.h>
14 using namespace std;

```

```

15
16 using ll = long long;
17
18 template <class T>
19 struct SegTree {
20     int n;
21     T identity;
22     function<T(T, T)> combine;
23     vector<T> t;
24
25     SegTree(int n, T identity, function<T(T, T)> combine)
26         : n(n), identity(identity), combine(move(combine)), t(2 * n, identity) {}
27
28     SegTree(const vector<T> &a, T identity, function<T(T, T)> combine)
29         : n((int)a.size()), identity(identity), combine(move(combine)),
30           t(2 * (int)a.size(), identity) {
31         for (int i = 0; i < n; i++) t[n + i] = a[i];
32         for (int i = n - 1; i > 0; i--) t[i] = this->combine(t[2 * i], t[2 * i + 1]);
33     }
34
35     // a[i] = value
36     void set(int i, T value) {
37         for (t[i += n] = value; i > 1; i >= 1)
38             t[i >> 1] = combine(t[i & ~1], t[i | 1]);
39     }
40
41     T get(int i) const { return t[i + n]; }
42
43     // Combine over [l, r) – half-open.
44     T query(int l, int r) const {
45         T resl = identity, resr = identity;
46         for (l += n, r += n; l < r; l >= 1, r >= 1) {
47             if (l & 1) resl = combine(resl, t[l++]);
48             if (r & 1) resr = combine(t[--r], resr);
49         }
50         return combine(resl, resr);
51     }
52 };
53
54 #ifdef TEST_SEGREE
55 int main() {
56     vector<ll> a = {5, 1, 3, 9, 2};
57
58     SegTree<ll> sum((int)a.size(), 0LL, [](ll x, ll y) { return x + y; });
59     for (int i = 0; i < (int)a.size(); i++) sum.set(i, a[i]);
60     assert(sum.query(0, 5) == 20);
61     assert(sum.query(1, 4) == 13); // [1,4) = 1+3+9
62     assert(sum.query(2, 3) == 3);
63     assert(sum.query(3, 3) == 0); // empty range
64
65     const ll INF = numeric_limits<ll>::max();
66     SegTree<ll> mn(a, INF, [](ll x, ll y) { return min(x, y); });
67     assert(mn.query(0, 5) == 1);
68     assert(mn.query(2, 5) == 2);
69     assert(mn.query(3, 4) == 9);
70     assert(mn.query(1, 1) == INF); // empty -> identity
71
72     mn.set(1, 100); // a[1]: 1 -> 100
73     assert(mn.query(0, 5) == 2);
74     assert(mn.get(1) == 100);
75
76     SegTree<ll> mx(a, numeric_limits<ll>::min(), [](ll x, ll y) { return max(x, y); });
77     assert(mx.query(0, 5) == 9);
78     assert(mx.query(0, 2) == 5);
79
80     // gcd works too -- any associative operation does.
81     vector<ll> g = {12, 18, 24};
82     SegTree<ll> gg(g, 0LL, [](ll x, ll y) { return __gcd(x, y); });
83     assert(gg.query(0, 3) == 6);
84     assert(gg.query(0, 2) == 6);
85     assert(gg.query(1, 3) == 6);
86
87     // Overflow guard.
88     vector<ll> big(1000, 1'000'000'000LL);
89     SegTree<ll> bs(big, 0LL, [](ll x, ll y) { return x + y; });
90     assert(bs.query(0, 1000) == 1'000'000'000'000LL);
91     puts("segment tree ok");
92 }
93 #endif

```

4.18 templates/cpp/graphs/traversal.cpp

```

1 // BFS and DFS on an unweighted graph, plus grid BFS. O(n + m).
2 //
3 // WHY THIS FILE EXISTS: KACTL names BFS/DFS only in its "Techniques" appendix
4 // and never implements them. These are the workhorses of NCPC's easy graph
5 // problems.
6 //
7 // TRAPS:
8 // * Recursive DFS stack-overflows around ~1e5 depth on a path graph.
9 //   Use the iterative version for large inputs.
10 // * BFS gives shortest path ONLY when every edge has the same weight.
11 // * In grids, bounds-check BEFORE indexing.
12 #include <bits/stdc++.h>
13 using namespace std;
14
15 // Shortest distances in edges from src; -1 if unreachable.
16 vector<int> bfs(const vector<vector<int>> &adj, int src) {
17     vector<int> dist(adj.size(), -1);
18     queue<int> q;
19     dist[src] = 0;

```

```

20     q.push(src);
21     while (!q.empty()) {
22         int u = q.front();
23         q.pop();
24         for (int v : adj[u])
25             if (dist[v] == -1) {
26                 dist[v] = dist[u] + 1;
27                 q.push(v);
28             }
29     }
30     return dist;
31 }
32
33 // Iterative DFS – safe at any depth. Returns visit order.
34 vector<int> dfs(const vector<vector<int>> &adj, int src) {
35     vector<char> seen(adj.size(), 0);
36     vector<int> order, st{src};
37     seen[src] = 1;
38     while (!st.empty()) {
39         int u = st.back();
40         st.pop_back();
41         order.push_back(u);
42         for (int v : adj[u])
43             if (!seen[v]) {
44                 seen[v] = 1;
45                 st.push_back(v);
46             }
47     }
48     return order;
49 }
50
51 // Number of connected components.
52 int countComponents(const vector<vector<int>> &adj) {
53     vector<char> seen(adj.size(), 0);
54     int c = 0;
55     for (size_t i = 0; i < adj.size(); i++)
56         if (!seen[i]) {
57             c++;
58             for (int u : dfs(adj, (int)i)) seen[u] = 1;
59         }
60     return c;
61 }
62
63 // Grid BFS from (sr,sc); '#' is a wall. Returns -1 for unreachable cells.
64 vector<vector<int>> gridBFS(const vector<string> &g, int sr, int sc) {
65     int R = (int)g.size(), C = (int)g[0].size();
66     vector<vector<int>> dist(R, vector<int>(C, -1));
67     const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
68     queue<pair<int, int>> q;
69     dist[sr][sc] = 0;
70     q.emplace(sr, sc);
71     while (!q.empty()) {
72         auto [r, c] = q.front();
73         q.pop();
74         for (int k = 0; k < 4; k++) {
75             int nr = r + dr[k], nc = c + dc[k];
76             if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue; // bounds FIRST
77             if (g[nr][nc] == '#' || dist[nr][nc] != -1) continue;
78             dist[nr][nc] = dist[r][c] + 1;
79             q.emplace(nr, nc);
80         }
81     }
82     return dist;
83 }
84
85 #ifdef TEST_TRAVERSAL
86 int main() {
87     // 0-1-2 3-4 5
88     vector<vector<int>> adj(6);
89     auto add = [&](int a, int b) { adj[a].push_back(b); adj[b].push_back(a); };
90     add(0, 1); add(1, 2); add(3, 4);
91
92     auto d = bfs(adj, 0);
93     assert(d[0] == 0 && d[1] == 1 && d[2] == 2);
94     assert(d[3] == -1 && d[5] == -1);
95     assert((int)dfs(adj, 0).size() == 3);
96     assert(countComponents(adj) == 3);
97
98     // Grid: start top-left, a wall forces a detour, and 'X' is sealed off.
99     vector<string> g = {
100         "...",
101         ".##.",
102         ".#X.",
103         "...";
104     auto gd = gridBFS(g, 0, 0);
105     assert(gd[0][0] == 0);
106     assert(gd[0][3] == 3); // straight along the top row
107     assert(gd[3][0] == 3); // straight down the left column
108     assert(gd[2][2] == 6); // 'X' is reachable, but only the long way around
109     assert(gd[1][1] == -1); // wall
110     puts("traversal ok");
111 }
112 #endif

```

4.19 templates/cpp/graphs/dijkstra.cpp

```

1 // Dijkstra – single-source shortest path, non-negative weights. O(m log n).
2 //
3 // WHY THIS FILE EXISTS: KACTL omits Dijkstra on purpose. Its README says it
4 // excludes "algorithms that are very common/simple (e.g., Dijkstra)" – it
5 // assumes you have it memorised. Carry it on paper.

```

```

6 //
7 // TRAPS:
8 // * dist must be long long. Weights up to 1e9 with 1e5 edges overflows int.
9 // * Use the lazy variant below (push duplicates, skip stale pops). Do not
10 //   try to decrease-key; it is not worth the code under time pressure.
11 // * Unreachable nodes keep dist == INF. Check before printing.
12 #include <bits/stdc++.h>
13 using namespace std;
14
15 using ll = long long;
16 const ll INF = numeric_limits<ll>::max() / 4; // /4 leaves room for d + w
17
18 // adj[u] = {v, weight}
19 vector<ll> dijkstra(const vector<vector<pair<int, ll>>& adj, int src) {
20     vector<ll> dist(adj.size(), INF);
21     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
22
23     dist[src] = 0;
24     pq.emplace(0, src);
25
26     while (!pq.empty()) {
27         auto [d, u] = pq.top();
28         pq.pop();
29         if (d != dist[u]) continue; // stale entry
30         for (auto [v, w] : adj[u]) {
31             if (d + w < dist[v]) {
32                 dist[v] = d + w;
33                 pq.emplace(dist[v], v);
34             }
35         }
36     }
37     return dist;
38 }
39
40 #ifdef TEST_DIJKSTRA
41 int main() {
42     //      (1)      (2)
43     // 0 ----> 1 ----> 2
44     // |          ^
45     // +------(5)-----+ node 3 isolated
46     vector<vector<pair<int, ll>>& adj(4);
47     auto add = [&](int a, int b, ll w) {
48         adj[a].emplace_back(b, w);
49         adj[b].emplace_back(a, w);
50     };
51     add(0, 1, 1);
52     add(1, 2, 2);
53     add(0, 2, 5);
54
55     auto d = dijkstra(adj, 0);
56     assert(d[0] == 0);
57     assert(d[1] == 1);
58     assert(d[2] == 3); // 0->1->2 beats the direct edge of 5
59     assert(d[3] == INF); // unreachable
60
61     // Overflow guard: a long path of 1e9 weights must not wrap.
62     int n = 1000;
63     vector<vector<pair<int, ll>>& big(n);
64     for (int i = 0; i + 1 < n; i++) {
65         big[i].emplace_back(i + 1, 1'000'000'000LL);
66         big[i + 1].emplace_back(i, 1'000'000'000LL);
67     }
68     auto db = dijkstra(big, 0);
69     assert(db[n - 1] == 999LL * 1'000'000'000LL);
70     puts("Dijkstra ok");
71 }
72 #endif

```

4.20 templates/cpp/graphs/algorithms.cpp

```

1 // More graph algorithms: topological sort, cycle detection, bipartite check,
2 // Floyd-Marshall, and Kruskal MST.
3 //
4 // NCPC uses graphs more than any other category (14 of 57 problems,
5 // 2021–2025), but almost always as "BFS/Dijkstra plus a twist" rather than
6 // an exotic algorithm. These are the twists that actually show up.
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 using ll = long long;
11
12 // Topological order of a DAG (Kahn). Returns {} if the graph has a cycle,
13 // which doubles as cycle detection for directed graphs.
14 vector<int> toposort(const vector<vector<int>>& adj) {
15     int n = (int)adj.size();
16     vector<int> indeg(n, 0), order;
17     for (int u = 0; u < n; u++)
18         for (int v : adj[u]) indeg[v]++;
19
20     priority_queue<int, vector<int>, greater<>> q; // smallest-first = deterministic
21     for (int u = 0; u < n; u++)
22         if (!indeg[u]) q.push(u);
23
24     while (!q.empty()) {
25         int u = q.top();
26         q.pop();
27         order.push_back(u);
28         for (int v : adj[u])
29             if (--indeg[v] == 0) q.push(v);
30     }
31     return (int)order.size() == n ? order : vector<int>{};

```

```

32 }
33
34 // Undirected cycle detection via DFS, iterative (no stack overflow).
35 bool hasCycleUndirected(const vector<vector<int>> &adj) {
36     int n = (int)adj.size();
37     vector<int> parent(n, -2);
38     for (int s = 0; s < n; s++) {
39         if (parent[s] != -2) continue;
40         vector<int> st{s};
41         parent[s] = -1;
42         while (!st.empty()) {
43             int u = st.back();
44             st.pop_back();
45             for (int v : adj[u]) {
46                 if (v == parent[u]) continue;
47                 if (parent[v] != -2) return true; // already seen -> cycle
48                 parent[v] = u;
49                 st.push_back(v);
50             }
51         }
52     }
53     return false;
54 }
55
56 // 2-colour the graph. Returns {} if it is not bipartite (has an odd cycle).
57 vector<int> bipartite(const vector<vector<int>> &adj) {
58     vector<int> colour(adj.size(), -1);
59     for (size_t s = 0; s < adj.size(); s++) {
60         if (colour[s] != -1) continue;
61         colour[s] = 0;
62         queue<int> q;
63         q.push((int)s);
64         while (!q.empty()) {
65             int u = q.front();
66             q.pop();
67             for (int v : adj[u]) {
68                 if (colour[v] == -1) {
69                     colour[v] = colour[u] ^ 1;
70                     q.push(v);
71                 } else if (colour[v] == colour[u]) {
72                     return {};
73                 }
74             }
75         }
76     }
77     return colour;
78 }
79
80 // All-pairs shortest paths. O(n^3) -- only for n <= ~400.
81 // d[i][j] = INF means unreachable. Handles negative edges (no negative cycle).
82 const ll INF = numeric_limits<ll>::max() / 4;
83
84 void floydWarshall(vector<vector<ll>> &d) {
85     size_t n = d.size();
86     for (size_t k = 0; k < n; k++)
87         for (size_t i = 0; i < n; i++)
88             if (d[i][k] < INF)
89                 for (size_t j = 0; j < n; j++)
90                     if (d[k][j] < INF) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
91 }
92
93 // Minimum spanning tree weight (Kruskal). Returns -1 if the graph is not
94 // connected. Edges are {weight, u, v}.
95 ll kruskal(int n, vector<array<ll, 3>> edges) {
96     sort(edges.begin(), edges.end());
97     vector<int> parent(n);
98     iota(parent.begin(), parent.end(), 0);
99     function<int(int)> find = [&](int x) {
100         while (parent[x] != x) x = parent[x] = parent[parent[x]];
101         return x;
102     };
103
104     ll total = 0;
105     int used = 0;
106     for (auto [w, u, v] : edges) {
107         int a = find((int)u), b = find((int)v);
108         if (a == b) continue;
109         parent[a] = b;
110         total += w;
111         used++;
112     }
113     return used == n - 1 ? total : -1;
114 }
115
116 #ifdef TEST_GRAPHS2
117 int main() {
118     // DAG: 0->1->3, 0->2->3
119     vector<vector<int>> dag(4);
120     dag[0] = {1, 2};
121     dag[1] = {3};
122     dag[2] = {3};
123     auto order = toposort(dag);
124     assert(order.size() == 4);
125     assert(order[0] == 0 && order[3] == 3);
126
127     // Add a cycle 3->0 and it must report failure.
128     dag[3].push_back(0);
129     assert(toposort(dag).empty());
130
131     // Undirected: path has no cycle, triangle does.
132     vector<vector<int>> path(3);
133     auto addU = [&](vector<vector<int>> &g, int a, int b) {
134         g[a].push_back(b);
135         g[b].push_back(a);
136     };

```

```

136 };
137 addU(path, 0, 1);
138 addU(path, 1, 2);
139 assert(!hasCycleUndirected(path));
140 addU(path, 2, 0);
141 assert(hasCycleUndirected(path));
142
143 // Even cycle is bipartite, odd cycle is not.
144 vector<vector<int>> even(4);
145 addU(even, 0, 1); addU(even, 1, 2); addU(even, 2, 3); addU(even, 3, 0);
146 auto col = bipartite(even);
147 assert(!col.empty() && col[0] == col[2] && col[0] != col[1]);
148
149 vector<vector<int>> odd(3);
150 addU(odd, 0, 1); addU(odd, 1, 2); addU(odd, 2, 0);
151 assert(bipartite(odd).empty());
152
153 // Floyd-Warshall on 0->1 (1), 1->2 (2), 0->2 (5); node 3 isolated.
154 vector<vector<ll>> d(4, vector<ll>(4, INF));
155 for (int i = 0; i < 4; i++) d[i][i] = 0;
156 d[0][1] = 1; d[1][2] = 2; d[0][2] = 5;
157 floydWarshall(d);
158 assert(d[0][2] == 3); // via 1
159 assert(d[0][3] == INF); // unreachable
160 assert(d[2][0] == INF); // directed
161
162 // MST: triangle with weights 1,2,3 -> 1+2 = 3
163 assert(kruskal(3, {{1, 0, 1}, {2, 1, 2}, {3, 0, 2}}) == 3);
164 assert(kruskal(3, {{1, 0, 1}}) == -1); // disconnected
165 assert(kruskal(1, {}) == 0); // single node
166 puts("graphs2 ok");
167 }
168 #endif

```

4.21 templates/cpp/dp/classic.cpp

```

1 // Classic DP patterns. DP is the most common named technique at NCPC
2 // (10 of 57 problems, 2021-2025), but it is almost always a variation on
3 // one of these shapes rather than a textbook recurrence.
4 //
5 // HOW TO RECOGNISE DP: you are asked for a count, a minimum/maximum, or
6 // "is it possible", AND the problem has a natural "first i items" or
7 // "prefix of the string" structure with overlapping subproblems.
8 //
9 // TRAPS:
10 // * Use long long for counts -- they explode.
11 // * Initialise carefully: 0, -infinity and "impossible" are different.
12 // * Loop ORDER decides 0/1 knapsack vs unbounded. See below.
13 #include <bits/stdc++.h>
14 using namespace std;
15
16 using ll = long long;
17
18 // 0/1 knapsack: each item used at most once. O(n * capacity).
19 // Iterate capacity DOWNWARD so each item is counted once.
20 ll knapsack01(const vector<int> &weight, const vector<ll> &value, int capacity) {
21     vector<ll> dp(capacity + 1, 0);
22     for (size_t i = 0; i < weight.size(); i++)
23         for (int c = capacity; c >= weight[i]; c--)
24             dp[c] = max(dp[c], dp[c - weight[i]] + value[i]);
25     return dp[capacity];
26 }
27
28 // Unbounded knapsack: unlimited copies. Same code, capacity UPWARD.
29 ll knapsackUnbounded(const vector<int> &weight, const vector<ll> &value, int capacity) {
30     vector<ll> dp(capacity + 1, 0);
31     for (size_t i = 0; i < weight.size(); i++)
32         for (int c = weight[i]; c <= capacity; c++)
33             dp[c] = max(dp[c], dp[c - weight[i]] + value[i]);
34     return dp[capacity];
35 }
36
37 // Can we hit exactly `target`? Classic subset-sum.
38 bool subsetSum(const vector<int> &a, int target) {
39     vector<char> dp(target + 1, 0);
40     dp[0] = 1;
41     for (int x : a)
42         for (int c = target; c >= x; c--)
43             if (dp[c - x]) dp[c] = 1;
44     return dp[target] != 0;
45 }
46
47 // Fewest coins summing to target; -1 if impossible.
48 ll coinChangeMin(const vector<int> &coins, int target) {
49     const ll BIG = numeric_limits<ll>::max() / 4;
50     vector<ll> dp(target + 1, BIG);
51     dp[0] = 0;
52     for (int c = 1; c <= target; c++)
53         for (int coin : coins)
54             if (coin <= c && dp[c - coin] != BIG)
55                 dp[c] = min(dp[c], dp[c - coin] + 1);
56     return dp[target] == BIG ? -1 : dp[target];
57 }
58
59 // Number of ways to make target where order does NOT matter.
60 // Coins must be the OUTER loop; swapping the loops counts permutations.
61 ll coinChangeWays(const vector<int> &coins, int target) {
62     vector<ll> dp(target + 1, 0);
63     dp[0] = 1;
64     for (int coin : coins)
65         for (int c = coin; c <= target; c++) dp[c] += dp[c - coin];

```

```

66     return dp[target];
67 }
68
69 // Longest increasing subsequence LENGTH,  $O(n \log n)$ .
70 // For non-decreasing, use upper_bound instead of lower_bound.
71 int lis(const vector<ll> &a) {
72     vector<ll> tails;
73     for (ll x : a) {
74         auto it = lower_bound(tails.begin(), tails.end(), x);
75         if (it == tails.end()) tails.push_back(x);
76         else *it = x;
77     }
78     return (int)tails.size();
79 }
80
81 // Longest common subsequence length.  $O(n*m)$ .
82 int lcs(const string &a, const string &b) {
83     vector<vector<int>> dp(a.size() + 1, vector<int>(b.size() + 1, 0));
84     for (size_t i = 1; i <= a.size(); i++)
85         for (size_t j = 1; j <= b.size(); j++)
86             dp[i][j] = (a[i - 1] == b[j - 1])
87                 ? dp[i - 1][j - 1] + 1
88                 : max(dp[i - 1][j], dp[i][j - 1]);
89     return dp[a.size()][b.size()];
90 }
91
92 // Edit distance (Levenshtein).  $O(n*m)$ .
93 int editDistance(const string &a, const string &b) {
94     vector<vector<int>> dp(a.size() + 1, vector<int>(b.size() + 1, 0));
95     for (size_t i = 0; i <= a.size(); i++) dp[i][0] = (int)i;
96     for (size_t j = 0; j <= b.size(); j++) dp[0][j] = (int)j;
97     for (size_t i = 1; i <= a.size(); i++)
98         for (size_t j = 1; j <= b.size(); j++)
99             dp[i][j] = (a[i - 1] == b[j - 1])
100                 ? dp[i - 1][j - 1]
101                 : 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]});
102     return dp[a.size()][b.size()];
103 }
104
105 // Maximum subarray sum (Kadane),  $O(n)$ . Requires at least one element,
106 // so an all-negative array returns its largest element.
107 ll maxSubarray(const vector<ll> &a) {
108     ll best = numeric_limits<ll>::min(), cur = 0;
109     for (ll x : a) {
110         cur = max(x, cur + x);
111         best = max(best, cur);
112     }
113     return best;
114 }
115
116 #ifdef TEST_DP
117 int main() {
118     vector<int> w = {2, 3, 4, 5};
119     vector<ll> v = {3, 4, 5, 6};
120     assert(knapsack01(w, v, 5) == 7); // 2+3 -> 3+4
121     assert(knapsack01(w, v, 0) == 0);
122     assert(knapsackUnbounded(w, v, 6) == 9); // 2+2+2 -> 3+3+3
123
124     assert(subsetSum({3, 34, 4, 12, 5, 2}, 9));
125     assert(!subsetSum({3, 34, 4, 12, 5, 2}, 30));
126     assert(subsetSum({1}, 0));
127
128     assert(coinChangeMin({1, 5, 10, 25}, 30) == 2);
129     assert(coinChangeMin({5, 10}, 3) == -1);
130     assert(coinChangeMin({2}, 0) == 0);
131     assert(coinChangeWays({1, 2, 5}, 5) == 4); // 11111, 1112, 122, 5
132     assert(coinChangeWays({2}, 3) == 0);
133
134     assert(lis({10, 9, 2, 5, 3, 7, 101, 18}) == 4);
135     assert(lis({}) == 0);
136     assert(lis({7}) == 1);
137     assert(lis({5, 4, 3}) == 1);
138
139     assert(lcs("ABCBDBAB", "BDCABA") == 4);
140     assert(lcs("abc", "abc") == 3);
141     assert(lcs("abc", "") == 0);
142
143     assert(editDistance("kitten", "sitting") == 3);
144     assert(editDistance("", "abc") == 3);
145     assert(editDistance("same", "same") == 0);
146
147     assert(maxSubarray({-2, 1, -3, 4, -1, 2, 1, -5, 4}) == 6);
148     assert(maxSubarray({-5, -2, -8}) == -2);
149     // Overflow guard: 1e5 values of 1e9 sums to 1e14.
150     assert(maxSubarray(vector<ll>(100000, 1'000'000'000LL)) == 100'000'000'000'000LL);
151     puts("dp ok");
152 }
153 #endif

```

4.22 templates/cpp/math/number_theory.cpp

```

1 // Elementary number theory – the kind NCPC actually uses.
2 //
3 // Across NCPC 2021–2025, number theory appeared in 9 of 57 problems, and it
4 // was almost always ELEMENTARY: gcd/lcm, modular arithmetic, parity, divisors,
5 // primes. Nothing exotic. This file covers that range.
6 //
7 // TRAPS:
8 // * lcm(a,b) overflows easily: divide BEFORE multiplying.
9 // * C++'s % returns a NEGATIVE result for negative input. Use mod() below.
10 // * modpow needs __int128 (or careful casting) if the modulus exceeds ~3e9.

```

```

11 #include <bits/stdc++.h>
12 using namespace std;
13
14 using ll = long long;
15
16 // C++17 has std::gcd / std::lcm in <numeric>. Hand versions for paper use:
17 ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
18
19 // Divide first – a*b would overflow for a,b near 1e9.
20 ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }
21
22 // Always–nonnegative modulo. C++'s % gives -1 for (-1 % 5).
23 ll mod(ll a, ll m) { return ((a % m) + m) % m; }
24
25 // (base^exp) % m in O(log exp).
26 ll modpow(ll base, ll exp, ll m) {
27     base = mod(base, m);
28     ll result = 1;
29     while (exp > 0) {
30         if (exp & 1) result = (ll)((__int128)result * base % m);
31         base = (ll)((__int128)base * base % m);
32         exp >>= 1;
33     }
34     return result;
35 }
36
37 // Modular inverse, for PRIME m only (Fermat's little theorem).
38 ll modinv(ll a, ll m) { return modpow(a, m - 2, m); }
39
40 // Extended Euclid: returns g = gcd(a,b) and sets x, y with a*x + b*y = g.
41 ll extgcd(ll a, ll b, ll &x, ll &y) {
42     if (!b) {
43         x = 1;
44         y = 0;
45         return a;
46     }
47     ll x1, y1;
48     ll g = extgcd(b, a % b, x1, y1);
49     x = y1;
50     y = x1 - (a / b) * y1;
51     return g;
52 }
53
54 // General modular inverse – works for ANY m coprime to a, not just prime m.
55 // Returns -1 when no inverse exists (gcd(a,m) != 1).
56 ll modinvGeneral(ll a, ll m) {
57     ll x, y;
58     if (extgcd(mod(a, m), m, x, y) != 1) return -1;
59     return mod(x, m);
60 }
61
62 // Smallest prime factor sieve up to n. Also gives primality and factorisation.
63 vector<int> sieve(int n) {
64     vector<int> spf(n + 1);
65     iota(spf.begin(), spf.end(), 0);
66     for (int i = 2; (ll)i * i <= n; i++)
67         if (spf[i] == i)
68             for (int j = i * i; j <= n; j += i)
69                 if (spf[j] == j) spf[j] = i;
70     return spf;
71 }
72
73 // Prime factorisation by trial division, O(sqrt(n)). Fine up to ~1e12.
74 vector<pair<ll, int>> factor(ll n) {
75     vector<pair<ll, int>> f;
76     for (ll p = 2; p * p <= n; p++) {
77         if (n % p) continue;
78         int e = 0;
79         while (n % p == 0) n /= p, e++;
80         f.emplace_back(p, e);
81     }
82     if (n > 1) f.emplace_back(n, 1); // leftover prime
83     return f;
84 }
85
86 // All divisors of n, unsorted. O(sqrt(n)).
87 vector<ll> divisors(ll n) {
88     vector<ll> d;
89     for (ll i = 1; i * i <= n; i++)
90         if (n % i == 0) {
91             d.push_back(i);
92             if (i != n / i) d.push_back(n / i);
93         }
94     return d;
95 }
96
97 bool isPrime(ll n) {
98     if (n < 2) return false;
99     for (ll p = 2; p * p <= n; p++)
100         if (n % p == 0) return false;
101     return true;
102 }
103
104 #ifdef TEST_NUMBER_THEORY
105 int main() {
106     assert(gcd(12, 18) == 6);
107     assert(gcd(17, 5) == 1);
108     assert(lcm(4, 6) == 12);
109     // Overflow guard: naive a*b/gcd would wrap, dividing first does not.
110     assert(lcm(1'000'000'000LL, 999'999'998LL) == 499999999000000000LL);
111
112     // C++ native % would give -1 here.
113     assert(mod(-1, 5) == 4);
114     assert(mod(-13, 5) == 2);

```

```

115  assert(mod(13, 5) == 3);
116
117  assert(modpow(2, 10, 1'000'000'007) == 1024);
118  assert(modpow(2, 0, 7) == 1);
119  // Large modulus: plain 64-bit multiply would overflow, __int128 does not.
120  assert(modpow(123456789, 123456789, 1'000'000'007LL) == 907408795LL);
121  assert(modinv(3, 7) == 5 && 3 * 5 % 7 == 1);
122
123  // Extended Euclid: the Bezout identity must actually hold.
124  ll x, y;
125  assert(extgcd(240, 46, x, y) == 2);
126  assert(240 * x + 46 * y == 2);
127  // General inverse works where Fermat's does not: 10 is composite.
128  assert(modinvGeneral(3, 10) == 7 && 3 * 7 % 10 == 1);
129  assert(modinvGeneral(3, 7) == 5); // agrees with Fermat on primes
130  assert(modinvGeneral(4, 10) == -1); // gcd(4,10)=2, no inverse exists
131
132  auto spf = sieve(50);
133  assert(spf[2] == 2 && spf[3] == 3 && spf[4] == 2 && spf[49] == 7);
134  int primes = 0;
135  for (int i = 2; i <= 50; i++)
136      if (spf[i] == i) primes++;
137  assert(primes == 15); // primes below 50
138
139  auto f = factor(360); // 2^3 * 3^2 * 5
140  assert(f.size() == 3);
141  assert(f[0] == make_pair(2LL, 3));
142  assert(f[1] == make_pair(3LL, 2));
143  assert(f[2] == make_pair(5LL, 1));
144  assert(factor(999'999'937LL).size() == 1); // large prime
145
146  auto d = divisors(36);
147  sort(d.begin(), d.end());
148  assert((d == vector<ll>{1, 2, 3, 4, 6, 9, 12, 18, 36}));
149
150  assert(isPrime(2) && isPrime(999'999'937LL));
151  assert(!isPrime(1) && !isPrime(0) && !isPrime(561));
152  puts("number theory ok");
153 }
154 #endif

```

4.23 templates/cpp/strings/strings.cpp

```

1 // String algorithms. Strings are uncommon at NCPC (4 of 57 problems,
2 // 2021-2025) and when they appear it is usually hashing or a simple scan ---
3 // not suffix automata. These cover that realistic range.
4 //
5 // TRAPS:
6 // * Hashing: use a RANDOM base. Fixed bases get anti-hash tested.
7 // * Hashing collisions are possible; use a 64-bit modulus.
8 // * KMP's failure function is 0-indexed here; off-by-ones are easy.
9 #include <bits/stdc++.h>
10 using namespace std;
11
12 using ll = long long;
13 using ull = unsigned long long;
14
15 // Prefix function (KMP): pi[i] = length of the longest proper prefix of
16 // s[0..i] that is also a suffix of it. O(n).
17 vector<int> prefixFunction(const string &s) {
18     vector<int> pi(s.size(), 0);
19     for (size_t i = 1; i < s.size(); i++) {
20         int j = pi[i - 1];
21         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
22         if (s[i] == s[j]) j++;
23         pi[i] = j;
24     }
25     return pi;
26 }
27
28 // All start positions of `pat` inside `text`, 0-indexed. O(n * m).
29 vector<int> findAll(const string &text, const string &pat) {
30     if (pat.empty() || pat.size() > text.size()) return {};
31     string combined = pat + '\x01' + text;
32     vector<int> pi = prefixFunction(combined), res;
33     for (size_t i = pat.size() + 1; i < combined.size(); i++)
34         if (pi[i] == (int)pat.size())
35             res.push_back((int)(i - 2 * pat.size()));
36     return res;
37 }
38
39 // Z-function: z[i] = length of the longest common prefix of s and s[i..].
40 vector<int> zFunction(const string &s) {
41     int n = (int)s.size();
42     vector<int> z(n, 0);
43     if (n) z[0] = n;
44     for (int i = 1, l = 0, r = 0; i < n; i++) {
45         if (i < r) z[i] = min(r - i, z[i - l]);
46         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
47         if (i + z[i] > r) l = i, r = i + z[i];
48     }
49     return z;
50 }
51
52 // Polynomial rolling hash with O(1) substring queries.
53 // Random base chosen at construction defeats anti-hash tests.
54 struct StringHash {
55     static const ull MOD = (1ULL << 61) - 1; // Mersenne prime
56     ull base;
57     vector<ull> h, p;
58

```

```

59     static ull mulmod(ull a, ull b) { return (ull)((__uint128_t)a * b % MOD); }
60
61     explicit StringHash(const string &s) {
62         static mt19937_64 rng(
63             (ull)chrono::steady_clock::now().time_since_epoch().count());
64         base = uniform_int_distribution<ull>(256, MOD - 2)(rng);
65
66         h.assign(s.size() + 1, 0);
67         p.assign(s.size() + 1, 1);
68         for (size_t i = 0; i < s.size(); i++) {
69             h[i + 1] = (mulmod(h[i], base) + (ull)s[i]) % MOD;
70             p[i + 1] = mulmod(p[i], base);
71         }
72     }
73
74     // Hash of s[l..r] inclusive, 0-indexed.
75     ull get(int l, int r) const {
76         return (h[r + 1] + MOD - mulmod(h[l], p[r - l + 1])) % MOD;
77     }
78 };
79
80 bool isPalindrome(const string &s) {
81     for (size_t i = 0, j = s.size(); i + 1 < j; i++, j--)
82         if (s[i] != s[j - 1]) return false;
83     return true;
84 }
85
86 // Split on a single delimiter. Keeps empty fields, like Python's str.split(c).
87 vector<string> split(const string &s, char delim) {
88     vector<string> out;
89     string cur;
90     istringstream iss(s);
91     while (getline(iss, cur, delim)) out.push_back(cur);
92     return out;
93 }
94
95 #ifdef TEST_STRINGS
96 int main() {
97     auto pi = prefixFunction("aabaab");
98     assert((pi == vector<int>{0, 1, 0, 1, 2, 2, 3}));
99     assert(prefixFunction("").empty());
100
101     assert((findAll("ababab", "ab") == vector<int>{0, 2, 4}));
102     assert((findAll("aaaa", "aa") == vector<int>{0, 1, 2}));
103     assert(findAll("abc", "xyz").empty());
104     assert(findAll("abc", "").empty());
105     assert(findAll("a", "abc").empty());
106
107     auto z = zFunction("aabxaab");
108     assert(z[0] == 7 && z[1] == 1 && z[4] == 3);
109     assert(zFunction("").empty());
110
111     string s = "abracadabra";
112     StringHash hs(s);
113     assert(hs.get(0, 3) == hs.get(7, 10)); // "abra" twice
114     assert(hs.get(0, 0) != hs.get(1, 1)); // 'a' vs 'b'
115     assert(hs.get(0, 10) == hs.get(0, 10));
116
117     // Same substring at different offsets must hash equal.
118     string t = "xxabcxxabc";
119     StringHash ht(t);
120     assert(ht.get(2, 4) == ht.get(7, 9));
121
122     assert(isPalindrome("racecar"));
123     assert(isPalindrome("abba"));
124     assert(isPalindrome(""));
125     assert(isPalindrome("a"));
126     assert(!isPalindrome("abc"));
127
128     assert((split("a,b,c", ',') == vector<string>{"a", "b", "c"}));
129     assert((split("a", ',') == vector<string>{"a"}));
130     assert((split("a,,b", ',') == vector<string>{"a", "", "b"}));
131     puts("strings ok");
132 }
133 #endif

```

4.24 templates/cpp/geometry/geometry.cpp

```

1 // Computational geometry. NCPC uses geometry heavily for a contest of its
2 // size (9 of 57 problems, 2021–2025), and convex hull alone appeared three
3 // times in five years.
4 //
5 // TRAPS:
6 // * Prefer INTEGER coordinates. cross() on long long is exact; doubles
7 //   introduce comparison bugs you will not find under time pressure.
8 // * cross(a,b,c) > 0 means c is LEFT of the directed line a->b.
9 // * Polygon area via the shoelace formula is TWICE the area; it is even,
10 //   so halving is exact — but keep it doubled while summing.
11 // * Collinear points: decide explicitly whether the hull should keep them.
12 #include <bits/stdc++.h>
13 using namespace std;
14
15 using ll = long long;
16
17 struct P {
18     ll x, y;
19     P operator-(const P &o) const { return {x - o.x, y - o.y}; }
20     bool operator<(const P &o) const { return tie(x, y) < tie(o.x, o.y); }
21     bool operator==(const P &o) const { return x == o.x && y == o.y; }
22 };
23

```

```

24 // Cross product of OA x OB. Sign gives orientation:
25 // > 0 counter-clockwise (b is left of o->a)
26 // = 0 collinear
27 // < 0 clockwise
28 ll cross(const P &a, const P &b, const P &c) {
29     return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
30 }
31
32 ll dot(const P &a, const P &b, const P &c) {
33     return (a.x - o.x) * (b.x - o.x) + (a.y - o.y) * (b.y - o.y);
34 }
35
36 ll dist2(const P &a, const P &b) {
37     ll dx = a.x - b.x, dy = a.y - b.y;
38     return dx * dx + dy * dy;
39 }
40
41 // TWICE the signed area of a polygon (shoelace). Positive if the vertices
42 // are counter-clockwise. Doubling keeps it exact in integers.
43 ll area2(const vector<P> &poly) {
44     ll s = 0;
45     for (size_t i = 0, n = poly.size(); i < n; i++) {
46         const P &a = poly[i], &b = poly[(i + 1) % n];
47         s += a.x * b.y - b.x * a.y;
48     }
49     return s;
50 }
51
52 // Convex hull, counter-clockwise, Andrew's monotone chain. O(n log n).
53 // Collinear points on the hull edges are EXCLUDED (strict '<= 0').
54 // To keep them, change to '< 0'.
55 vector<P> convexHull(vector<P> pts) {
56     sort(pts.begin(), pts.end());
57     pts.erase(unique(pts.begin(), pts.end()), pts.end());
58     int n = (int)pts.size();
59     if (n < 3) return pts;
60
61     vector<P> h(2 * n);
62     int k = 0;
63     for (int i = 0; i < n; i++) {
64         while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
65         h[k++] = pts[i];
66     }
67     for (int i = n - 2, t = k + 1; i >= 0; i--) {
68         while (k >= t && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
69         h[k++] = pts[i];
70     }
71     h.resize(k - 1);
72     return h;
73 }
74
75 // Is point q inside (or on the border of) a polygon? Ray casting, O(n).
76 // Works for any simple polygon, convex or not.
77 bool pointInPolygon(const vector<P> &poly, const P &q) {
78     bool inside = false;
79     for (size_t i = 0, n = poly.size(), j = n - 1; i < n; j = i++) {
80         const P &a = poly[i], &b = poly[j];
81         // On the boundary?
82         if (cross(a, b, q) == 0 && dot(q, a, b) <= 0) return true;
83         if ((a.y > q.y) != (b.y > q.y)) {
84             // Does the ray to the right cross edge a-b?
85             ll t = cross(a, b, q);
86             if ((b.y > a.y) ? t > 0 : t < 0) inside = !inside;
87         }
88     }
89     return inside;
90 }
91
92 // Do segments p1p2 and p3p4 intersect (including touching)?
93 bool segmentsIntersect(const P &p1, const P &p2, const P &p3, const P &p4) {
94     auto sgn = [](ll v) { return (v > 0) - (v < 0); };
95     int d1 = sgn(cross(p3, p4, p1)), d2 = sgn(cross(p3, p4, p2));
96     int d3 = sgn(cross(p1, p2, p3)), d4 = sgn(cross(p1, p2, p4));
97     if (d1 * d2 < 0 && d3 * d4 < 0) return true;
98     // Collinear touching cases.
99     auto onSeg = [](const P &a, const P &b, const P &c) {
100         return cross(a, b, c) == 0 && min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
101             min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
102     };
103     return onSeg(p3, p4, p1) || onSeg(p3, p4, p2) || onSeg(p1, p2, p3) ||
104         onSeg(p1, p2, p4);
105 }
106
107 #ifndef TEST_GEOMETRY
108 int main() {
109     P o{0, 0}, a{1, 0}, b{0, 1};
110     assert(cross(o, a, b) > 0); // counter-clockwise
111     assert(cross(o, b, a) < 0); // clockwise
112     assert(cross(o, a, P{2, 0}) == 0); // collinear
113     assert(dist2(o, P{3, 4}) == 25);
114
115     // Unit square, counter-clockwise: area 1 -> area2 == 2
116     vector<P> sq = {{0, 0}, {1, 0}, {1, 1}, {0, 1}};
117     assert(area2(sq) == 2);
118     reverse(sq.begin(), sq.end());
119     assert(area2(sq) == -2); // clockwise flips the sign
120     reverse(sq.begin(), sq.end());
121
122     // 10x10 square with an interior point: hull must drop the interior one.
123     vector<P> pts = {{0, 0}, {10, 0}, {10, 10}, {0, 10}, {5, 5}};
124     auto h = convexHull(pts);
125     assert(h.size() == 4);
126     assert(area2(h) == 200);
127 }

```

```

128 // Collinear points along an edge are excluded.
129 auto h2 = convexHull({{0, 0}, {1, 0}, {2, 0}, {2, 2}, {0, 2}});
130 assert(h2.size() == 4);
131
132 // Degenerate inputs must not crash.
133 assert(convexHull({{1, 1}}).size() == 1);
134 assert(convexHull({{0, 0}, {1, 1}}).size() == 2);
135 assert(convexHull({{3, 3}, {3, 3}}).size() == 1); // duplicates removed
136
137 vector<P> poly = {{0, 0}, {4, 0}, {4, 4}, {0, 4}};
138 assert(pointInPolygon(poly, {2, 2})); // inside
139 assert(pointInPolygon(poly, {0, 0})); // vertex
140 assert(pointInPolygon(poly, {2, 0})); // on an edge
141 assert(!pointInPolygon(poly, {5, 2})); // outside
142 assert(!pointInPolygon(poly, {-1, -1}));
143
144 assert(segmentsIntersect({0, 0}, {2, 2}, {0, 2}, {2, 0})); // crossing
145 assert(segmentsIntersect({0, 0}, {2, 0}, {1, 0}, {3, 0})); // overlapping
146 assert(segmentsIntersect({0, 0}, {1, 1}, {1, 1}, {2, 2})); // touching
147 assert(!segmentsIntersect({0, 0}, {1, 1}, {2, 2}, {3, 3})); // collinear apart
148 assert(!segmentsIntersect({0, 0}, {1, 0}, {0, 1}, {1, 1})); // parallel
149
150 // Overflow guard: coordinates near 1e9 make cross() reach ~1e18,
151 // which fits long long but would destroy int.
152 P big1{0, 0}, big2{1'000'000'000LL, 0}, big3{0, 1'000'000'000LL};
153 assert(cross(big1, big2, big3) == 1'000'000'000'000'000'000LL);
154 puts("geometry ok");
155 }
156 #endif

```

4.25 templates/python/solution.py

```

1 """Python solution template for NCPC 2026.
2
3 THE JUDGE RUNS PYPY3 7.3.23 == Python 3.11.15, not CPython.
4
5 * Do NOT use syntax newer than 3.11. Your local Python is 3.14 and will
6 accept things the judge rejects.
7 * numpy is slow or absent under PyPy. Plain loops are FAST under PyPy.
8 * sys.stdin.buffer.read() is mandatory; input() is far too slow.
9
10 USE PYTHON WHEN: the problem is string/bigint/parsing heavy, or the logic is
11 fiddly and n is small. USE C++ WHEN: n is large or the limit is tight.
12
13 Local run:
14 python3 sol.py < a.in > a.out
15 LOCAL=1 python3 sol.py # reads in.txt, writes out.txt
16 """
17 import os
18 import sys
19
20 # Debug to stderr, never stdout -- stdout is the answer the judge reads.
21 DEBUG = os.environ.get("LOCAL") == "1"
22
23
24 def dbg(*args):
25     if DEBUG:
26         print(*args, file=sys.stderr)
27
28
29 def main():
30     data = sys.stdin.buffer.read().split()
31     pos = 0
32
33     def nxt_int():
34         nonlocal pos
35         pos += 1
36         return int(data[pos - 1])
37
38     def nxt_str():
39         nonlocal pos
40         pos += 1
41         return data[pos - 1].decode()
42
43     out = []
44
45     # ---- solve ----
46     n = nxt_int()
47     a = [nxt_int() for _ in range(n)]
48     dbg("n =", n, "a =", a)
49     out.append(str(sum(a)))
50     # -----
51
52     sys.stdout.write("\n".join(out) + "\n")
53
54
55 if DEBUG and os.path.exists("in.txt"):
56     sys.stdin = open("in.txt")
57     sys.stdout = open("out.txt", "w")
58
59 main()

```